

Prompts, System Instructions & Development Workflow Documentation

Track 1 — KOHLER AI Bathroom Designer & Planner

Submission	Individual Case Study — Track 1
Solution	AI Bathroom Designer & Planner (single-page web app)
Document purpose	Full record of the prompts, system instructions and iterative workflow used
Date	September 20, 2026

This document reconstructs, in order, the prompting workflow used to take the KOHLER AI Bathroom Designer & Planner from a blank page to the working prototype submitted with this entry. It is organised into three parts: (1) the phased prompt log used to direct an AI coding assistant through each stage of the build, (2) the single runtime AI system instruction that ships inside the working product itself (the inspiration-photo analyser), and (3) a summary of where AI was used versus where deterministic, rule-based logic was used and why.

Part 1 — Development Workflow & Prompt Log

The application was built conversationally with an AI coding assistant across a series of focused sessions, one per subsystem. Each session started from a specific, scoped prompt describing the feature and its constraints (physical, budget, or aesthetic), and was refined through 2-4 follow-up prompts once the first pass was reviewed in the browser. The prompts below are reconstructed from that working process and grouped into the phases they correspond to in the shipped code.

Phase 0 — Problem framing

Prompt — Initial brief

"I want to build an AI bathroom designer for KOHLER: the user enters room length, width, height, a budget in INR, and picks an aesthetic theme (Minimalist Modern, Classic Luxury, Japanese Zen, Urban Industrial, Coastal Spa, Scandi Warm). The app should generate three complete, physically valid bathroom layouts automatically, each with a real KOHLER product bundle that fits the space and the budget, and show them in both a 2D floor plan and a 3D room. Start by helping me scope the data model and the algorithm before we write any UI."

Outcome: Produced the initial architecture split: a fixture/product catalogue, a set of aesthetic "packs" (palette + materials + product IDs per theme), a geometric placement solver operating in inches, and a rendering layer (2D canvas plan + 3D scene) sitting on top of one shared layout object.

Phase 1 — Data modelling

Prompt — Catalogue & aesthetics

"Define a product catalogue for toilets, showers, faucets, vanities, mirrors and bathtubs with realistic KOHLER-style names, categories, specs and INR prices. Then define six aesthetic packs, each mapping to a distinct palette, wall/floor material pattern, metal finish, and a specific product from the catalogue for every fixture type, so swapping the aesthetic swaps the entire bundle consistently."

Outcome: Generated the PRODUCTS catalogue and the AES object (six themes, each carrying its own colour palette, procedural material description, and a full products{} map).

Prompt — Real-world placement rules

"Add real fixture footprint dimensions in inches, and encode Vastu Shastra directional placement guidance as a scoring function — given a compass bearing and a fixture kind, return how favourable that wall/position is, so the solver can use it as one of several scoring signals rather than a hard rule."

Outcome: Produced the fixture spec table and vastuScore(kind, bearing), later surfaced to the user as a per-design Vastu compliance score.

Phase 2 — Layout solver (the recommendation engine)

Prompt — Core constraint solver

"This is the core of the product: write a solver that, given room L×W×H, a door wall, and a list of wanted fixtures, finds a non-overlapping arrangement that (a) respects each fixture's required clearance, (b) keeps a walkable path from the door to every fixture, (c) never blocks the door swing, and (d) prefers wall positions that face the doorway or align with plumbing walls. Score every candidate position per fixture and greedily pack the best-scoring combination, falling back to a smaller fixture size before dropping a fixture entirely."

Outcome: Produced the candidate-generation/scoring/greedy-pack pipeline: door candidate search with swing-collision checks, per-wall candidate placement, a multi-factor scoreCandidate() (clearance, Vastu bearing, adjacency, door-facing), an occupancy grid for reachability/circulation checks, and size-fallback packing.

Prompt — Accessories, budget and validation

"Once the main fixtures are placed, automatically add supporting accessories — mirror above the basin, towel rail near the bathing zone, paper holder and bin within reach of the toilet, robe hook by the door, ceiling fan and light — using the free space that's left. Then compute total cost against the user's budget, validate the whole layout for any leftover overlap or blocked circulation, and produce a short plain-language Vastu report the user can read."

Outcome: Produced `placeAccessories()`, `costLayout()`, `validate()` and `vastuReport()`, plus the top-level `solve()` that tries multiple door positions/swing directions per aesthetic and keeps the three highest-scoring, distinct results to present as the three generated designs.

Phase 3 — Visualising the layout

Prompt — 2D floor plan

"Render the solved layout as a clean top-down 2D floor plan on canvas — walls, door swing arc, and a distinct symbol for every fixture and accessory, styled to match the selected aesthetic's palette."

Outcome: Produced `drawPlan()` and the per-fixture `planSymbol()/accSymbol()` renderers.

Prompt — 3D room

"Build a 3D version of the same layout using Three.js: procedurally generate the wall/floor/feature materials per aesthetic (tile, marble, stone, slat patterns) instead of using image textures, so everything stays self-contained with no asset downloads, build simple parametric geometry for each fixture type, add an environment map for realistic reflections, and give the user orbit-style camera controls plus drag-to-reposition for movable items."

Outcome: Produced the procedural material/canvas generators, geometry helper functions (rounded rect, ellipse, extrude, box/cylinder/tube), a per-aesthetic material builder, individual model builders per fixture, and the camera control rig — all seeded deterministically so a given layout always renders identically.

Phase 4 — Product & comparison pages

Prompt — Catalogue browsing

"Add a separate product catalogue page with category filter chips, a card per product showing an icon, specs and price, an 'add to design' action, and a running cart tray with item count and subtotal."

Outcome: Produced the CATS filter chips, the catalogue card renderer, and the cart tray logic (addbtn handlers, running total).

Prompt — Design comparison

"Add a compare view that puts the three generated designs side by side, one column each, listing which product was chosen per fixture category, the total cost, and the AI match / space-fit / Vastu scores for each."

Outcome: Produced `paintCompare()` and its comparison table.

Phase 5 — Conversational design assistant

Prompt — Rule-based chat layer

"Add a chat panel next to the plan where the user can ask things like 'what's within my budget', 'why can't you add a bathtub', or 'swap the vanity for something cheaper', and get an answer grounded in the actual solved layout object — not a generic chatbot reply. Match intents with pattern rules against the real layout data (fixtures placed, notices raised during solving, cost breakdown) rather than calling a language model for every message, so answers are fast, free and always consistent with what's on screen."

Outcome: Produced the `respond()` dispatcher: a set of intent patterns tested against the user's message, each pulling its answer directly from the live layout/state object (`STATE.layout`, `L.notices`, cost breakdown) rather than free-form generation.

Phase 6 — Inspiration-photo feature

Prompt — Bridging a photo to the solver

"Let the user upload a photo of a bathroom they like. Send it to the AI vision model with a strict instruction to return only a JSON object describing style, colours, tiles, flooring, finishes and which fixtures are visible. Map the returned style to the closest of my six aesthetic presets and to any fixtures the user hasn't already added, then re-run the exact same solver used everywhere else in the app so the result stays physically valid — never let the AI's suggestion override the room's real constraints."

Outcome: Produced the inspiration upload/preview flow, the `inspoPrompt()` system instruction documented in full in Part 2, and `inspoApply()`, which translates the returned JSON into aesthetic + wants changes and calls the same `solve()` used by manual input.

Prompt — Extending the chat assistant without touching it

"Once there's an inspiration analysis, let the same chat panel answer questions like 'match my inspiration', 'what can I add from my inspiration', 'why couldn't you copy X', and 'show me matching products' — but wire this in as an add-on layer in front of the existing dispatcher rather than editing its internals, so the original logic can't regress."

Outcome: Produced `inspoRespond()` and the wrapper that calls it before falling through to the original `respond()`, keeping the two concerns isolated.

Phase 7 — Landing page, theming & polish

Prompt — Brand-appropriate front end

"Design a KOHLER-appropriate landing page above the tool itself: a serif display headline, brass/porcelain colour tokens, a hero room render, scroll-reveal animations, and full light/dark theme support via CSS custom properties, matching both the OS theme and a manual toggle."

Outcome: Produced the token system (`:root` variables, `prefers-color-scheme` + `data-theme` overrides), the hero section, scroll-reveal/marquee/ripple micro-interactions, and the theme toggle.

Prompt — Responsiveness pass

"Do a pass for small laptop screens specifically — the planner's three-panel layout (controls, plan/3D, chat) needs to stay usable without scrolling on a 13-inch display, without changing any of the underlying logic."

Outcome: Produced the compact-layout CSS override block, applied as pure presentation changes on top of the existing structure.

Phase 8 — Hardening & edge cases

Prompt — Failure-mode review

"Walk through what happens when the room is too small for the requested fixtures, when no door position has a valid swing, when the budget can't fit even the cheapest bundle, and when the AI vision call fails or times out — each case needs a clear, specific message to the user, never a silent failure or a generic error."

Outcome: Produced the notices/warn system surfaced in the layout report, the `solve()` fallback ordering, and the full inspiration-analysis error-code map (rate limits, malformed response, unsupported image, capability unavailable, etc.), each with its own user-facing copy.

Part 2 — Runtime AI System Instruction

Everything in Part 1 describes prompts used *while building* the app. The feature below is different: it is the one point where the shipped, running application itself calls a vision-capable AI model live, using a fixed system instruction embedded in the code. This is the only place the product depends on a model call at runtime — every other piece of intelligence (the layout solver, budget optimisation, Vastu scoring, and the chat assistant) is deterministic logic that runs entirely client-side, described in Part 3.

Feature: Inspiration Photo Analyser

When the user uploads a reference bathroom photo and presses **Analyze with AI**, the app sends the image to a connected vision-capable model together with the exact system instruction below, requesting a single strict JSON object and nothing else. The instruction is reproduced verbatim from the source (function `inspoPrompt()`):

```
You are analyzing a photo of a bathroom for an interior-design planning tool.
Look only at the attached image. Reply with ONLY one JSON object, no other text, no markdown fences, matching exactly this shape:
{
  "style": string,                // your own short label for the overall style, e.g. "Japanese Zen", "Industrial Loft"
  "closestPreset": string,       // exactly one of: "modern", "luxury", "zen", "industrial", "coastal", "scandi"
                                  //   modern=Minimalist Modern, luxury=Classic Luxury, zen=Japanese Zen,
                                  //   industrial=Urban Industrial, coastal=Coastal Spa, scandi=Scandi Warm - pick whichever is closest
  "colors": [string, string, string?], // 2-3 dominant colors, short names like "Beige", "Matte Black"
  "tiles": string,
  "flooring": string,
  "vanity": string,              // short description, or "not visible" if none appears in the photo
  "toilet": string,             // short description, or "not visible"
  "shower": string,            // short description, or "not visible"
  "bathtub": string,           // short description, or "not visible"
  "mirror": string,            // short description, or "not visible"
  "lighting": string,
  "finishes": string,          // the dominant hardware/metal finish, e.g. "Matte black", "Brushed brass", "Chrome"
  "accessories": [string]      // short list, e.g. ["Towel rail", "Plant", "Storage niche"]
}
```

Why the prompt is designed this way

- Single strict JSON contract, no prose: the app parses the reply programmatically and feeds it straight into the layout pipeline, so any free text or markdown fencing would break the integration — the instruction explicitly forbids both.
- A closed enum for `closestPreset` (modern / luxury / zen / industrial / coastal / scandi) constrains the model's open-ended style read to exactly the six aesthetic packs the solver already knows how to build, so the output is always actionable rather than merely descriptive.
- Every fixture field explicitly allows "not visible" — the model is told not to guess fixtures that aren't in the photo, which keeps false positives out of the "what can I add from my inspiration" flow.
- The response never sets geometry, cost, or placement directly. It only ever changes the chosen aesthetic and the wanted-fixtures list, both of which are then run back through the same deterministic solver from Part 1 — so an AI misread of a photo can change style, never make the layout physically invalid or exceed the budget.

Failure handling

Because this is the only live model call in the product, it is also the only place that needs runtime failure handling. Each known failure mode (permission not granted, images unsupported in the current view, unreadable image, rate limiting, sampling disabled, malformed/invalid JSON returned, model refusal, empty response, upstream error) is mapped to its own specific, user-facing message rather than a generic "something went wrong".

Part 3 — Where AI Was Used vs. Deterministic Logic

For transparency, the table below separates every intelligent-seeming behaviour in the product into (a) AI-assisted during development, (b) a live model call at runtime, or (c) deterministic algorithmic logic that was written, with AI assistance, but runs without any model in the loop.

Capability	How it works	AI involvement
Layout generation (3 designs per request)	Constraint solver: candidate scoring, greedy packing, occupancy-grid reachability checks	Built via AI-assisted development prompts (Part 1). Runs deterministically, no model call.
Budget-fit optimisation	Cost totals computed from the fixed product catalogue against user budget; size/fixture fallback on overrun	Deterministic logic, AI-assisted build
Vastu compliance scoring	Compass-bearing lookup table scored per fixture kind	Deterministic logic, AI-assisted build
2D plan / 3D room rendering	Canvas 2D + Three.js procedural materials and geometry	Deterministic logic, AI-assisted build
Design chat assistant	Pattern-matched intents answered from the live layout object	Deterministic logic, AI-assisted build
Inspiration photo analysis	Uploaded image + fixed JSON-schema instruction sent to a vision-capable model; strict JSON parsed back	Live AI model call at runtime (Part 2)

Summary

The build workflow used an AI coding assistant as a pair-programmer across every layer of the stack — data modelling, the geometric solver, both renderers, the catalogue/compare pages, the chat assistant, and the UI — directed through the scoped, iterative prompts logged in Part 1. The shipped product carries that work forward as fast, deterministic, explainable logic, and reserves an actual live AI model call for the one task that genuinely needs open-ended perception rather than rules: reading an uploaded inspiration photo. That call is fully specified in Part 2, and its output is always constrained back through the same deterministic solver, so the AI can change what the room looks like but never make it physically or financially invalid.